

Nora: Tengo los requisitos funcionales para una aplicación en este documento. y los requisitos funcionales son:

1- Seguridad: Para identificar el tipos de usuario hace falta el login. Todos los usuarios estarán logeados para poder acceder a la aplicaión. Y podrán acceder a diferentes funcionalidades dependiendo del tipo de usuario.

2- Usuarios concurrentes: Si hay 20 usuario conectados el tiempo de respuesta de la aplicación será <1s.

Si hay 100 usuarios conectados el tiempo de respuesta de la aplicación será <5s.

Si hay más de 100 usuarios, la aplicación no va a funcionar.

El tiempo de respuesta aumentará progresivamente, es decir, si hay 40 usuarios conectados, la aplicación tardará más de 1s pero menos de 5s, según corresponda.

La aplicación tendrá el login usando SSO de la universidad.

La aplicación será una aplicación web.

La aplicación tendrá base de datos.

Se quiere montar todo esto usando AWS. Como harías la arquitectura teniendo todo esto en cuenta, usando los servicios disponibles en aws

ChatGPT: Montaría algo así (pensando en tu sistema de biblioteca con SSO y roles de estudiante/profesor/administrativo/bibliotecario):

1. Visión general de la arquitectura en AWS

Front-end (web)

- **Amazon S3** para servir la parte estática de la aplicación (HTML, CSS, JS).
- **Amazon CloudFront** delante de S3 para:

- Mejorar tiempos de respuesta globalmente (caché de contenido estático).
- Terminar HTTPS (certificado en **AWS Certificate Manager**).

Back-end (lógica de negocio)

- Opción muy razonable: **Amazon ECS Fargate** (contenedores serverless) o **AWS Elastic Beanstalk** (si quieres algo más “out of the box” para apps web).
- Desplegado en una **VPC** con subredes privadas.
- Acceso desde Internet a través de:
 - **Application Load Balancer (ALB)** en subredes públicas → reparte tráfico a las tareas/instancias del back-end.

Base de datos

- **Amazon RDS** (PostgreSQL / MySQL) en subredes privadas.
- Multi-AZ si quieres alta disponibilidad, Single-AZ si el presupuesto es más importante que la HA.
- Toda la lógica de préstamos, reservas, sanciones, etc., persiste aquí.

Cache

- **Amazon ElastiCache (Redis)** para:
 - Cachear consultas frecuentes (búsqueda de obras, disponibilidad...).
 - Guardar contadores de usuarios concurrentes y sesiones activas (para aplicar tu regla de “no más de 100 usuarios”).

Autenticación SSO universidad

- **Amazon Cognito** como “federador” de identidad:
 - Configurado como **Service Provider SAML** u **OIDC** frente al SSO de la universidad.
 - Cognito recibe el “tipo de usuario” (rol) como atributo del SSO (estudiante, profesor, bibliotecario...).
 - Devuelve al front/back un **JWT** con los *claims* de rol, sin que tu app maneje contraseñas.

Seguridad y configuración

- **AWS WAF** front al ALB/CloudFront para protección de ataques web básicos.
- **Security Groups** y **NACLs** para aislar:

- ALB en subred pública.
- ECS/Beanstalk y RDS en subredes privadas.
- **AWS Secrets Manager / SSM Parameter Store** para guardar:
 - Cadenas de conexión a RDS.
 - Secretos de integración con el SSO (client secret, certificados, etc.).

Monitorización y logs

- **Amazon CloudWatch** (métricas, logs, alarmas).
- **AWS X-Ray** opcional para trazas de peticiones y diagnóstico de rendimiento.

2. Flujo de autenticación y autorización por rol

1. Usuario accede a <https://biblioteca.uni.es> (CloudFront → S3 para estático y ALB para API).
2. El front detecta que no hay sesión y redirige a **Cognito**.
3. Cognito redirige al **SSO universitario** (SAML/OIDC).
4. Usuario se autentica con credenciales institucionales.
5. El SSO devuelve a Cognito una aserción con:
 - a. Identidad de usuario.
 - b. Atributos: tipo de usuario (estudiante, profesor, bibliotecario...), email, etc.
6. Cognito emite un **ID token / access token (JWT)** que el cliente web usará en cada llamada al back-end.
7. El back-end valida el JWT (clave pública de Cognito) y extrae el rol para:
 - a. Permitir consultas y reservas a todos.
 - b. Permitir préstamos/devoluciones/gestión de sanciones solo a bibliotecarios, etc.

3. Control de usuarios concurrentes y tiempos de respuesta

Requisito:

- ≤ 20 usuarios → tiempo de respuesta $< 1s$.

- ≤ 100 usuarios $\rightarrow$ tiempo de respuesta $< 5s$.
- 100 usuarios $\rightarrow$ la aplicación **no debe funcionar** (rechazar).

3.1 Cómo contar usuarios concurrentes

Usaría **ElastiCache (Redis)** como “fuente de verdad” de concurrencia:

- En el **back-end**, al autenticar un usuario (primer request tras login):
 - Añades una clave, por ejemplo:

```
active_sessions:<user_id> = ttl 15m
```

- O mantienes una estructura tipo SET con los usuarios activos.
- Cada vez que el usuario hace una petición relevante:
 - Renuevas el TTL de su sesión.
- También mantienes una clave `active_users_count` con el número total de usuarios únicos activos (puede derivarse del SET o mantenerse vía INCR/DECR).

3.2 Bloquear si hay más de 100 usuarios

- Antes de aceptar una nueva sesión (nuevo login) o nueva reserva/préstamo:
 - El back-end consulta `active_users_count` en Redis.
 - Si `count >= 100`, devuelves HTTP 429 Too Many Requests o 503 Service Unavailable con un mensaje tipo:

“Se ha alcanzado el límite máximo de usuarios concurrentes. Inténtalo de nuevo más tarde.”

Esto satisface explícitamente el requisito de “si hay más de 100 usuarios, la aplicación no va a funcionar”.

Si quieres ser más estricto, puedes:

- Implementar una **Lambda de pre-autenticación de Cognito** que consulte Redis y rechace el login si ya hay 100 usuarios activos.

3.3 Asegurar tiempos de respuesta

Para conseguir los tiempos indicados:

Diseño de rendimiento:

- **Front** en S3 + CloudFront → tiempos muy bajos para estático.
- **Backend:**
 - ECS Fargate con Service Auto Scaling (métricas CPU / latencia ALB).
 - Dimensionar al menos 2 tareas/instancias para 100 usuarios concurrentes (depende de carga, pero es más que suficiente para la mayoría de sistemas CRUD).
- **Base de datos RDS:**
 - Ajustar tamaño de instancia (por ejemplo db.t3.medium o superior según pruebas).
 - Índices bien diseñados en tablas de obras, ejemplares, reservas, préstamos, etc.
- **Caching con ElastiCache:**
 - Cachear resultados de búsquedas de catálogos (muy leídas, poco cambiantes).
 - Esto reduce queries a RDS y mejora latencia.

Naturalmente, el tiempo de respuesta se irá incrementando entre 20 y 100 usuarios a medida que suben las colas internas, cumpliendo tu requisito de aumento progresivo.

4. Arquitectura lógica a alto nivel (en texto)

Imagina los bloques así:

- **Internet**
 - ↓
- **CloudFront (HTTPS)**
 - Sitio estático desde **S3** (Front-end SPA).
 - API /api/* enrutada al **ALB**.
 - ↓
- **Application Load Balancer**
 - ↓
- **ECS Fargate / Elastic Beanstalk (Back-end en subred privada)**
 - Llama a:
 - **Cognito** para validar tokens (JWT).

- **ElastiCache (Redis)** para:
 - Contador de usuarios concurrentes.
 - Cache de consultas.
 - **RDS (PostgreSQL/MySQL)** para datos persistentes.
- Todo dentro de una **VPC**:
 - Subredes públicas: ALB, NAT Gateway.
 - Subredes privadas: ECS/Beanstalk, RDS, ElastiCache.

5. Relación con tus requisitos funcionales/no funcionales

- **Seguridad y SSO**
 - SSO de universidad integrado vía **Cognito + SAML/OIDC**, sin gestión de contraseñas propias.
 - Roles por tipo de usuario se obtienen como *claims* del IdP y se usan para permisos en el back-end.
- **Usuarios concurrentes y rendimiento**
 - Límite duro de 100 usuarios implementado vía Redis + lógica en back-end o pre-auth Lambda en Cognito.
 - Tiempos de respuesta optimizados con CloudFront + ECS escalable + RDS + ElastiCache.
- **Aplicación web con base de datos**
 - Web app servida por S3/CloudFront.
 - Base de datos relacional con RDS, persistente y gestionada.
- **Mantenibilidad y configuración**
 - Parámetros como tiempos de préstamo, reglas de sanciones y coeficientes se guardan en RDS y se exponen en una UI especial para bibliotecarios (admin).
 - Secretos y credenciales en Secrets Manager / SSM.

Si quieres, en el siguiente paso puedo:

- Bajar esto a un **diagrama concreto** (tipo C4 Nivel 2).

- O proponerte una **plantilla de infraestructura como código** (CloudFormation / Terraform) con los recursos básicos.

Nora: Porque no usarías aws lambda?

ChatGPT:

Buena pregunta, la respuesta corta es: **sí se puede usar Lambda**, pero antes te propuse ECS/Beanstalk porque encaja mejor con *este tipo concreto* de aplicación web “clásica” con SSO, sesiones y BBDD relacional.

Te resumo el porqué 📌

1. Retos de usar solo AWS Lambda para tu backend

Imagina la arquitectura “100% serverless”:

- CloudFront →
- **API Gateway** →
- **AWS Lambda** →
- RDS + ElastiCache

Funciona, pero tiene varias pegas para tus requisitos:

a) Límite de usuarios concurrentes (máx. 100)

Tú quieres:

- Hasta 100 usuarios → OK
- Más de 100 usuarios → **la app NO debe funcionar**

Con Lambda:

- Por defecto escala “sin límite” (hasta los límites de cuenta), justo lo contrario de lo que quieres.
- Se puede usar **concurrency limit / reserved concurrency** de Lambda para limitar invocaciones, pero:
 - Ese límite es por función, no “por usuario”.

- Empieza a devolver errores 429/5xx de forma menos controlada.
- Tienes menos control fino sobre el concepto de “usuario conectado”; realmente estás limitando peticiones simultáneas, no sesiones.

Con ECS/Beanstalk:

- Tú controlas mejor el **estado de sesiones activas** (en Redis) y decides explícitamente:
 - Si `active_users > 100` → devuelvo 429 con un mensaje amigable.
- No estás peleándote con el autoscaling de Lambda para emular un límite “duro” de usuarios.

b) Latencia y tiempos de respuesta

Requisito:

- ≤ 20 usuarios → **< 1s**
- ≤ 100 usuarios → **< 5s**

Lambda tiene:

- **Cold starts:** cuando la función no está “caliente”, el primer request puede tardar más (cientos de ms o más, según runtime y VPC).
- Si el tráfico es bajo o irregular (típico en una app universitaria fuera de horas punta), tendrás *más* cold starts, justo donde quieres <1s casi siempre.

Con ECS/Beanstalk:

- Tienes procesos siempre levantados.
- La latencia es más estable y predecible.
- Es más fácil “garantizar” ese <1s para 20 usuarios, porque no dependes del arranque de funciones.

c) Conexiones a la base de datos (RDS)

Con Lambda + RDS:

- Cada Lambda puede abrir conexiones nuevas → **connection storm** si hay picos.
- Casi obligado a usar **RDS Proxy** para manejar pooling.
- Más componentes, más configuración.

Con ECS/Beanstalk:

- Tienes pocos contenedores/instancias de backend con un pool de conexiones estable.
- Menos riesgo de saturar RDS.
- Arquitectura más sencilla de razonar.

d) Tipo de aplicación

Tu app es:

- Web “normalita” con:
 - Login SSO,
 - vistas,
 - lógica CRUD,
 - usuarios conectados.
- Seguramente un framework tipo Spring / Django / .NET / Node Express, etc.

Esto encaja muy bien en:

- **ECS Fargate** (contenerizado) o
- **Elastic Beanstalk** (si quieres despliegue sencillo “subo mi .jar/.zip y listo”).

Lambda brilla más cuando:

- Son **microservicios pequeños y bien aislados**,
- Mucha lógica **event-driven** (S3, colas, cron, etc.),
- Picos fuertes pero breves de tráfico,
- Y te compensa al máximo el modelo “pay per invocation”.

2. Entonces... ¿cuándo sí usaría Lambda aquí?

Yo la usaría, pero **no como backend principal**, sino para tareas auxiliares:

- Envío de emails de recordatorio (préstamo a punto de vencer).
- Jobs periódicos (cron con EventBridge) para:
 - Recalcular sanciones,

- Generar informes diarios/semanales.
- Procesar ficheros masivos (por ejemplo, carga de un catálogo desde CSV subido a S3).
- Integraciones puntuales con otros sistemas de la universidad.

Ahí Lambda es perfecto: ejecuta, hace su trabajo, se apaga y no pagas más.

3. Si aun así quisieras todo en Lambda...

Se puede plantear algo así:

- CloudFront + S3 (front)
- API Gateway (REST o HTTP)
- Lambda (funciones por recurso: /obras, /usuarios, /prestamos, etc.)
- Cognito + SSO uni para auth
- ElastiCache (Redis) para:
 - Contar usuarios concurrentes.
 - Cachear búsquedas.
- RDS + RDS Proxy para BBDD.

Pero:

- Tendrías que pelear más con:
 - Límite de concurrencia.
 - Cold starts.
 - Gestión de conexiones.
- Y no ganas tantísimo en coste si la app va a estar usada todos los días en horario de biblioteca.

Resumen

No es que *no se pueda* usar Lambda: es que, con tus requisitos concretos (límite duro de usuarios, tiempos de respuesta bastante estrictos y backend web clásico con BBDD relacional), **ECS/Beanstalk** te dan:

- Más control.
- Menos sorpresas de latencia.
- Arquitectura más simple de operar.

Si quieres, en el siguiente mensaje puedo dibujarte **las dos arquitecturas lado a lado (ECS vs Lambda)** y marcar pros/cons para que puedas defenderlo en una presentación o memoria de proyecto.

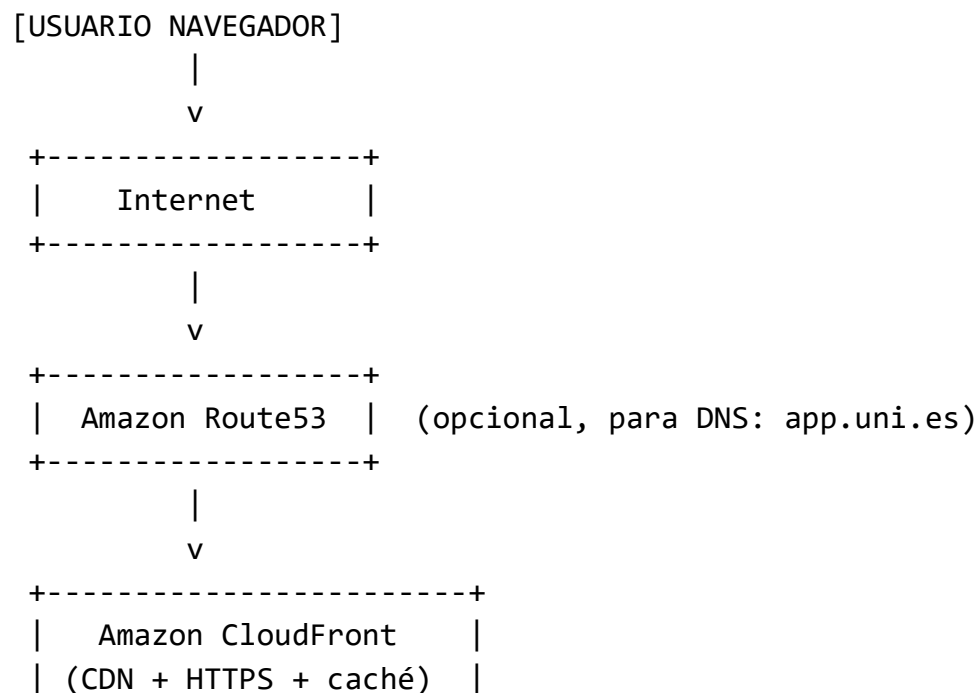
Nora: Quiero que hagas el esquema de la aplicación que me has propuesto sin usar Lambda. Házme un esquema sin usar imágenes como tal.

ChatGPT:

Perfecto, vamos a poner la arquitectura “limpita” y sin Lambda, sólo con lo que hablamos: S3 + CloudFront, ALB, ECS/Beanstalk, RDS, ElastiCache, Cognito, etc.

Lo hago en texto / ASCII, nada de imágenes reales 🙌

1. Esquema general por capas



- Amazon Cognito (SSO universidad)

- AWS Secrets Manager / Parameter Store
- Amazon CloudWatch (logs, métricas)

2. Detalle de cada bloque

2.1 Capa de acceso / presentación

USUARIO -> CloudFront -> S3 (front-end)

- **Amazon S3**
 - Aloja la aplicación web estática (SPA en React / Angular / Vue o similar).
- **Amazon CloudFront**
 - Distribuye el contenido estático con baja latencia.
 - Termina HTTPS (certificado en AWS Certificate Manager).
 - Puede enrutar:
 - / y ficheros estáticos → S3.
 - /api/* → ALB.

Opcionalmente, **Route 53** sólo para tener un dominio tipo biblioteca.uni.es.

2.2 Autenticación con SSO (sin Lambda)

Navegador

|

| 1) Redirección a login

v

Amazon Cognito <----> SSO Universidad (SAML / OIDC)

|

| 2) Devuelve JWT al front

v

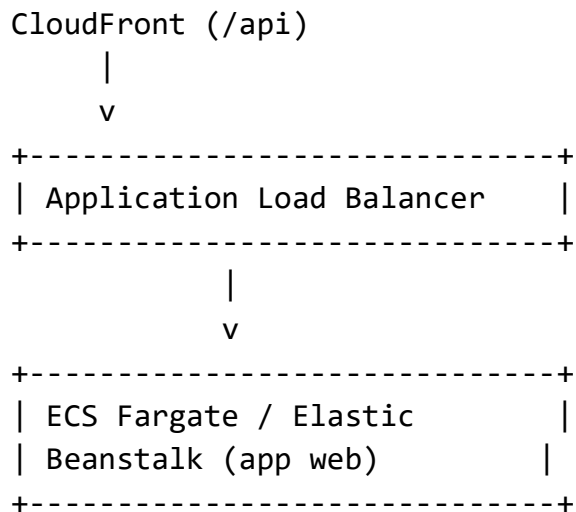
Back-end valida token en cada petición

- **Amazon Cognito** se configura como:

- “Federado” con el IdP de la universidad (SAML u OIDC).
 - Recibe los atributos del usuario (rol, email, etc.).
 - Devuelve un JWT que el front envía en las llamadas al back-end.
- El back-end (en ECS/Beanstalk) valida el token y usa los **claims** de rol para decidir qué puede hacer cada tipo de usuario.

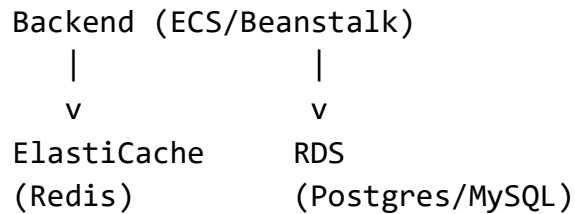
Importante: aquí no usamos Lambda en ningún punto, Cognito habla directamente con el IdP y con tu app.

2.3 Capa de balanceo y backend



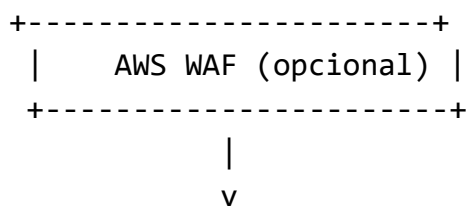
- **Application Load Balancer (ALB)**
 - Recibe tráfico HTTP/HTTPS desde CloudFront.
 - Reparte peticiones entre tareas ECS o instancias Beanstalk en subredes privadas.
- **ECS Fargate o Elastic Beanstalk**
 - Ejecutan tu app (por ejemplo: Spring Boot, Node.js, .NET, etc.).
 - Tienen acceso a:
 - **ElastiCache (Redis)** para cachear y contar usuarios concurrentes.
 - **RDS** para datos persistentes.
 - Se despliegan en **subredes privadas** de la VPC, sin acceso directo desde Internet.

2.4 Base de datos y caché



- **Amazon RDS:**
 - Base de datos relacional para:
 - Usuarios (o referencia al ID del SSO).
 - Libros / recursos.
 - Préstamos, reservas, multas, etc.
 - En subred privada.
 - Opción de Multi-AZ para alta disponibilidad.
- **Amazon ElastiCache (Redis):**
 - Caché de:
 - Resultados de búsquedas frecuentes.
 - Datos poco cambiantes.
 - Contador de **usuarios concurrentes**:
 - Claves de sesión activa por usuario.
 - Clave con `active_users_count`.
 - Antes de aceptar nuevas sesiones / operaciones, el backend comprueba:
 - Si `active_users_count >= 100` → responde con error controlado (la aplicación “no funciona” para más de 100).

2.5 Seguridad y configuración



ALB/CloudFront

- **AWS WAF:**
 - Protege de ataques típicos (SQL injection, XSS, etc.).
- **Security Groups / NACLs:**
 - ALB en subred pública.
 - ECS/Beanstalk, RDS, ElastiCache en subredes privadas.
 - Sólo los puertos necesarios abiertos:
 - ALB: 80/443 público.
 - ECS ↔ RDS, ECS ↔ ElastiCache: puertos internos.
- **AWS Secrets Manager / SSM Parameter Store:**
 - Almacenan:
 - Cadena de conexión a RDS.
 - Secretos de Cognito (client secret, certificados del IdP si hace falta).
 - Configuración sensible.

2.6 Observabilidad

ECS/Beanstalk, ALB, RDS, etc.

|

v

Amazon CloudWatch

(logs + métricas + alarmas)

- Logs de la app, del sistema, del ALB.
- Métricas:
 - Latencia de ALB (para comprobar esos <1s y <5s).
 - CPU / memoria de ECS/instancias Beanstalk.
 - Conexiones a RDS.
- Alarmas (por ejemplo, si la latencia media supera X ms, o si RDS está al 80% CPU).

3. Resumen textual muy compacto

Si lo quieres en “línea” para poner en una memoria:

Usuario → Internet → Route53 (DNS) → CloudFront (HTTPS, caché)
→ S3 (front web) y /api → WAF (opcional) → ALB
→ ECS Fargate / Elastic Beanstalk en VPC privada
→ (ElastiCache Redis para caché y control de usuarios concurrentes,
RDS para base de datos relacional).

Autenticación vía Cognito federado con SSO de la universidad.

Seguridad con Security Groups, WAF, Secrets Manager; monitorización con CloudWatch.

(No se usa AWS Lambda en esta arquitectura).

Si quieres, en el siguiente paso puedo ayudarte a convertir este esquema en una **sección “Arquitectura del Sistema”** lista para pegar en tu documento de proyecto (con numeración, texto más formal y todo eso).

Nora: Si

Chatgpt: Perfecto, te lo dejo listo para copiar/pegar en tu memoria como una sección formal. Adáptale solo la numeración (3.x, 4.x, etc.) según tu índice.

Arquitectura del sistema

1. Visión general

La solución propuesta se basa en una arquitectura en la nube sobre Amazon Web Services (AWS), orientada a una aplicación web con autenticación mediante SSO de la universidad, control explícito de usuarios concurrentes y uso de base de datos relacional.

Se ha optado **intencionadamente por no utilizar AWS Lambda** como plataforma de ejecución, priorizando en su lugar servicios como Amazon ECS Fargate o AWS Elastic Beanstalk para el backend. Esto permite un mayor control sobre el estado de la aplicación, las conexiones a base de datos, la gestión de sesiones y el cumplimiento del requisito de limitar el número de usuarios concurrentes.

La arquitectura se organiza en varias capas:

- Capa de presentación (front-end web).
- Capa de entrega y seguridad perimetral.
- Capa de backend (lógica de negocio).
- Capa de datos (base de datos y caché).
- Servicios transversales (autenticación, configuración y monitorización).

2. Componentes principales

2.1 Capa de presentación

- **Amazon S3**

La aplicación web (HTML, CSS, JavaScript) se despliega como contenido estático en un bucket de Amazon S3. Este bucket actúa como origen estático del front-end, facilitando la entrega de la interfaz de usuario de manera altamente disponible y escalable.

- **Amazon CloudFront**

Se utiliza CloudFront como red de distribución de contenido (CDN) para:

- Reducir la latencia mediante caché de los recursos estáticos.
- Terminar las conexiones HTTPS utilizando certificados gestionados en AWS Certificate Manager.
- Encaminamiento de peticiones:
 - Rutas estáticas (por ejemplo, / y ficheros de la SPA) hacia S3.
 - Rutas de API (por ejemplo, /api/...) hacia el balanceador de carga de la capa de backend.

Opcionalmente, el nombre de dominio público (por ejemplo, `app.universidad.es`) se gestiona mediante **Amazon Route 53** como servicio de DNS.

2.2 Autenticación y gestión de identidad

- **Amazon Cognito (federado con el SSO de la universidad)**

La autenticación de los usuarios se realiza mediante el sistema de Single Sign-On (SSO) de la universidad (SAML u OIDC). Para ello:

- Amazon Cognito actúa como proveedor de servicios (Service Provider), federado con el proveedor de identidad institucional.
- Tras el inicio de sesión, el SSO de la universidad devuelve a Cognito una aserción con la identidad y los atributos del usuario (incluyendo el tipo de usuario o rol).
- Cognito emite tokens JWT (ID token y/o access token) que el front-end incluye en cada petición a la API.
- El backend valida estos tokens en cada solicitud y extrae los **roles de usuario** (por ejemplo, estudiante, profesor, personal administrativo, etc.) para aplicar la autorización a nivel de lógica de negocio.

De esta forma, la aplicación **no gestiona directamente credenciales de usuario**, delegando la autenticación en el SSO institucional y utilizando Cognito como capa de federación.

2.3 Capa de entrega y seguridad perimetral

- **AWS WAF (opcional pero recomendable)**

Entre CloudFront y el Application Load Balancer se puede desplegar AWS WAF (Web Application Firewall) para:

- Proteger la aplicación frente a ataques comunes (SQL Injection, XSS, etc.).
- Definir reglas personalizadas de filtrado de tráfico.

- **Application Load Balancer (ALB)**

El ALB actúa como punto de entrada al backend:

- Recibe las peticiones HTTP/HTTPS procedentes de CloudFront.
- Las distribuye entre las instancias o tareas donde se ejecuta la aplicación backend (ECS Fargate o Elastic Beanstalk).
- Se despliega en subredes públicas dentro de una VPC, con asociaciones de **Security Groups** que restringen el tráfico únicamente a los puertos y orígenes necesarios.

2.4 Capa de backend (lógica de negocio)

La lógica de negocio de la aplicación se ejecuta en una capa de backend sin uso de AWS Lambda, utilizando uno de los siguientes servicios gestionados:

- **Amazon ECS Fargate**

Permite ejecutar contenedores Docker sin gestionar servidores. La aplicación backend se empaqueta como contenedor y se despliega en tareas ECS con Fargate como motor de cómputo.

- **AWS Elastic Beanstalk**

Alternativamente, se puede usar Beanstalk si se prefiere un despliegue más simplificado (subiendo directamente el artefacto de la aplicación, por ejemplo un `.jar` o un paquete Node.js), dejando a Beanstalk la gestión de la infraestructura subyacente.

En ambos casos:

- Las instancias/tareas del backend se despliegan en **subredes privadas** de la VPC.
- Solo reciben tráfico desde el ALB.
- Desde esta capa se accede a:
 - **Amazon RDS** (base de datos relacional).
 - **Amazon ElastiCache (Redis)** para caché y control de usuarios concurrentes.
- La aplicación valida los tokens JWT emitidos por Cognito y aplica las reglas de negocio y autorización por rol.

2.5 Capa de datos: base de datos y caché

- **Amazon RDS (PostgreSQL / MySQL)**

La base de datos relacional almacena de forma persistente toda la información necesaria para el sistema (usuarios o sus referencias, entidades del dominio, operaciones, etc.).

Características principales:

- Despliegue en subredes privadas dentro de la VPC.
- Posibilidad de configuración Multi-AZ para alta disponibilidad.
- Gestión de copias de seguridad automáticas, actualizaciones menores y monitorización.
- **Amazon ElastiCache (Redis)**

Se utiliza Redis con dos propósitos principales:

- **Caché de datos de lectura frecuente**
 - Resultados de consultas pesadas o muy recurrentes (por ejemplo, listados o búsquedas frecuentes).
 - Reducción de carga en RDS y mejora del tiempo de respuesta.
- **Control de usuarios concurrentes**
 - Se mantiene una estructura de datos en Redis que representa las sesiones activas o usuarios conectados.
 - El backend actualiza esta información cada vez que el usuario interactúa con la aplicación.
 - Se conserva un contador o conjunto de usuarios activos que permite conocer, en tiempo real, el número de usuarios concurrentes.
 - Antes de aceptar nuevas sesiones o peticiones críticas, la aplicación consulta Redis:
 - Si el número de usuarios concurrentes es **mayor de 100**, el backend devuelve una respuesta de error controlada, cumpliendo el requisito de que la aplicación **no funcione** cuando se exceda este límite.
 - Para valores intermedios (por ejemplo, entre 20 y 100 usuarios), el sistema sigue siendo funcional, si bien el tiempo de respuesta podrá aumentar progresivamente dentro de los márgenes establecidos (< 5 segundos).

2.6 Servicios transversales

- **AWS Systems Manager Parameter Store / AWS Secrets Manager**

Se emplean para almacenar de forma segura:

- Cadenas de conexión a la base de datos RDS.

- Credenciales y secretos de integración con el proveedor de identidad (SSO de la universidad).
- Cualquier otro parámetro de configuración sensible.
- **Amazon CloudWatch**

Todos los componentes (ALB, ECS/Beanstalk, RDS, etc.) envían métricas y logs a CloudWatch. A partir de esta información se pueden:

- Definir alarmas (por ejemplo, latencia media superior a un umbral, uso elevado de CPU o memoria, etc.).
- Monitorizar el cumplimiento de los requisitos de rendimiento:
 - Tiempo de respuesta medio inferior a 1 segundo con hasta 20 usuarios concurrentes.
 - Tiempo de respuesta medio inferior a 5 segundos con hasta 100 usuarios concurrentes.
- **Security Groups y NACLs**

La seguridad de red se implementa con:

- **Security Groups** para controlar el tráfico a nivel de instancia/servicio (ALB, backend, RDS, ElastiCache).
- **Listas de control de acceso a la red (NACLs)**, si es necesario, para restringir rangos de IP o puertos a nivel de subred.

3. Justificación de la no utilización de AWS Lambda

Aunque AWS Lambda es una tecnología adecuada para arquitecturas serverless y procesamiento basado en eventos, en este proyecto se ha optado por **no utilizarla** en la capa de backend principal, por los siguientes motivos:

- La aplicación es una **web transaccional “clásica”** con sesiones de usuario, SSO, y una base de datos relacional con acceso frecuente.
- Se requiere un **control explícito y determinista del número de usuarios concurrentes** y de la latencia de respuesta, lo cual se gestiona de forma más directa con procesos de backend residentes (ECS/Beanstalk) que con funciones efímeras.
- El uso de ECS Fargate o Elastic Beanstalk:

- Simplifica la gestión de conexiones a base de datos (pools estables).
- Evita los problemas de latencia por “cold starts” asociados a Lambda.
- Facilita la implementación de lógica de sesiones y control de concurrencia en Redis.

Lambda podría emplearse puntualmente para tareas asíncronas o por lotes (por ejemplo, envíos de correo programados, generación de informes, etc.), pero no se ha considerado necesaria para la arquitectura principal de la aplicación.

Si quieres, en otro mensaje te puedo ayudar a añadir una sección específica de **“Requisitos de rendimiento y escalabilidad”** donde encajes explícitamente los 20/100 usuarios y los tiempos de respuesta con esta arquitectura.